Shell Tutorial

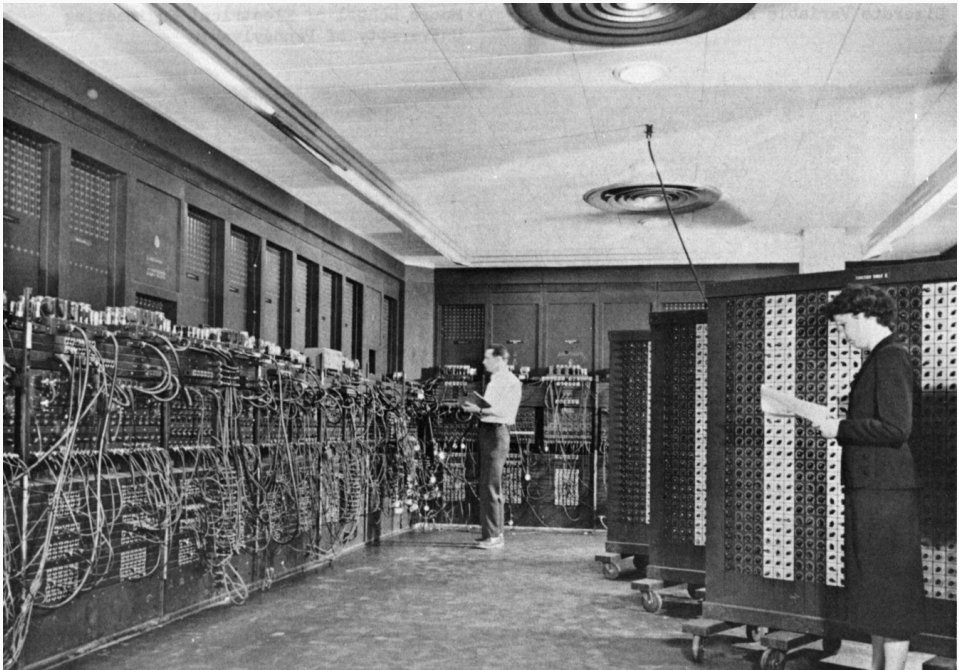
Computer programming 1

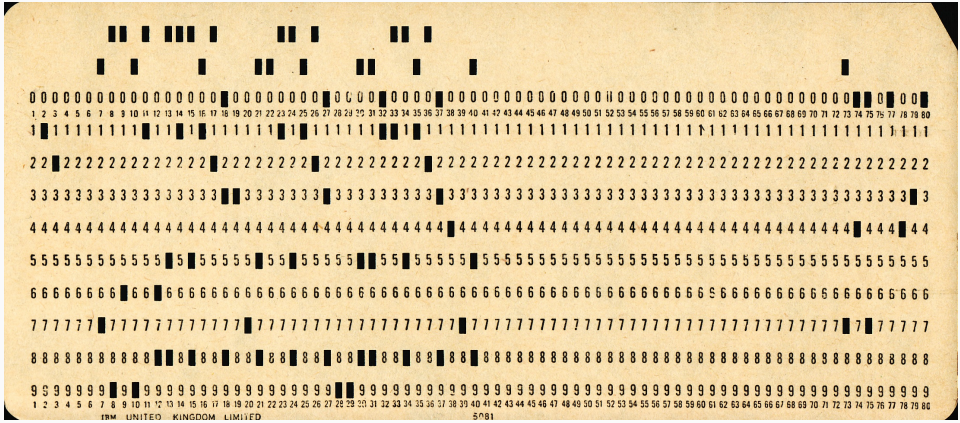
<2025-10-05 Sun>

Outline



How did the first computer input devices look like?





(Found on Wikipedia, author Pete Birkinshaw from Manchester, UK, CC2.0)


SLMIN .SYS 12P 20-Dec-85
XL .SYS 4P 20-Dec-85
SP .SYS 6P 13-Aug-86
RT11SJ.SYS 70P 13-Aug-86
SL .SYS 9P 13-Aug-86
SB .SYS 9P 13-Aug-86
BASIC .SAV 58 24-Mar-86
DATIME.SAV 4 20-Dec-85
DUMP .SAV 9 20-Dec-85
TSXMOD.SAV 70 27-Mar-86
MAKRIB.SAV 41 12-Jun-86
START .P36 2 21-Dec-85
EMSA .STH 19P 02-Feb-83
STAND .LIH 12P 15-Feb-83
EMIB .STH 19 11-Feb-83
JKFLP.FOR 3 08-Mar-86
ANSOT .SAV 30 15-Jul-83
DMO .DIR 1 16-Jul-86
DIR .DIR 26 16-Jul-86
DEMOMG.OBJ 1 0AP-
114 Files, 5949 Blocks
14433 Free blocks
.dg
VM .SYS 3P 13-Aug-86
FD .SYS 9P 13-Aug-86
MF .SYS 9P 13-Aug-86
DU .SYS 9P 13-Aug-86
TT .SYS 3P 13-Aug-86
RL02PC.SYS 71P 21-Jul-86
BINCON.SAV 24 20-Dec-85
DIR .SAV 19 20-Dec-85
DUP .SAV 47 20-Dec-85
FORTRA.SAV 206 21-Jul-86
LET .SAV 5 20-Dec-85
RETRO .OBJ 1515P 16-Jul-86
TSXUCL.TSK 22 07-Jul-86
IP 04-Sep-86
TSXV6 .MOD 3P 08-Jul-86
JKFLIP.SAV 30 08-Jul-86
RT11FB.SYS 93P 20-Dec-85
TSXP23.MCR 1209P 27-Mar-86
ML .DIR 7 16-Jul-86
DEMOFC.OBJ 1 0AP-
EVAN .ID 1
digital VT100

The Shell



- The shell provides a keyboard-driven interface to control and program the computer
- Functions available
  - Process control
  - File management
- The shell has its own programming language: you can either invoke one command at a time or write scripts which combine actions (very much like C++)
- Various shells available, among which `sh`, `bash`, `fish`, `zsh`

Advantages of the shell over a GUI



Precision, speed, programmability, repeatability

Advantages of the shell over a GUI



… and, sometimes, there is no GUI: container, servers

Emacs



- Text editor developed at MIT in the seventies
- Programmable and extensible in Lisp (and with macros)
- Provides various amenities for writing slides, generating websites, write programs, run shells, connect to remote computers, read email
- It runs on the terminal -> keep it in mind if you ever need to access a server with no GUI

Emacs



- Steep learning curve, but worthwhile
- Invoke with `emacs`
- Follow the tutorial `C-h t` (Control+h then t)

Navigating the File System



- You always operate on a directory (`pwd`)
- The file system is hierarchical (a directory contains other directories)
- You can inspect and move around with `ls` and `cd`

Example




```
cd /home/adolfo
# shortcuts
cd
cd ~
cd ..
cd -
pwd
ls
ls --color
```

Amenities



- The shell usually keeps a list of the commands you run (usually = unless you tell it not to). Navigate the history with up arrow or `CTR-R`
- `TAB` tries to complete, `*` matches all elements in the current directory (use it with extra care)
- Move in a command with `CTRL-b`, `CTRL-f`, `CTRL-a`, `CTRL-e`
- Empty a command line with `CTRL-k`

Controlling processes



- You can launch commands from the shell by typing their name
- *(The shell looks for executable files in the directories stored in the PATH variable, but this does not really matter, here)*
- Control jobs with:
  - `CTRL-C` kill foreground process
  - `CTRL-Z`, suspend foreground process
  - `CTRL-D` send EOF

Controlling processes




```
CTRL-Z   # interrupt foreground process
bg       # start in bg
fg       # put in fg
CTRL-Z
jobs     # what jobs is the shell controlling?
kill %1  # kill the first
ps auxw  # what jobs are running?
top      # check system resources
```

Combining processes: sequence



- Use `||` and `&&` to execute programs in sequence:

```
g++ a.cpp || ./a.out  # compile and then run, even when compil
g++ a.cpp && ./a.out  # compile and, if ok, run the program
```

Redirection and Pipes



- Most unix programs take input from `STDIN` and output to `STDOUT` and `STDERR`
- The shell allows to redirect, that is, change input and output sources
- The most common redirect operators are `>`, `<`, and `|` (pipe)

Redirection



Redirection allow to take input or store output from/to a file

```
# sort line from stdin (end with CTRL-D)
sort

# sort line from stdin save to file
sort > sorted.txt

# sort lines from unsorted and sort
sort < unsorted.txt > sorted.txt
```

Pipes



Pipes allow to chain commands (the output of a command becomes the input of the next command)

```
# show the /etc/password file
cat /etc/password

# ... and sort it
cat /etc/password | sort

# ... and extract the second column
cat /etc/password | sort | cut -f2 -d:

# ... and show only the uniq elements
cat /etc/password | sort | cut -f2 -d: | uniq
```

Useful commands



- `g++ file.cpp && ./a.out` Compile and execute if it compiles
- `find` Find files
- `grep` Search for content in files
- `man` Get information about a command
- `cat`, `more` (or `less`) output and page a file
- `cut` Operate on a tabular file, cut columns
- `bc` Perform math
- `tar` Create an archive